



Pet Room

Camilla Giuliani - 1883207- July 12th 2026

Repository name: **final-project-camydog_igproject**

GitHub Pages demo → https://sapienzainteractivegraphicscourse.github.io/final-project-camydog_igproject/

Project Overview

PetRoom is an interactive WebGL virtual pet project inspired by Nintendogs-style games.

The user can explore a 3D environment, interact with a skinned animated dog, switch between a home scene and a park scene, and test several real-time interactions such as feeding, giving water, calling and petting the dog, playing with a ball, throwing a frisbee and controlling a teapot chase minigame.

The project combines computer graphics techniques, user interaction, simple game logic, physics simulation, shadow mapping and animated objects. The goal is to create a small but complete interactive environment where the user can control the scene and observe how the dog reacts to different actions.

Project information is also included in the README file in the GitHub repository, here https://github.com/SapienzaInteractiveGraphicsCourse/final-project-camydog_igproject

Development environment & libraries

I used JavaScript and **WebGL 1.0**, without using high-level rendering engines such as Three.js or Babylon.js. The rendering pipeline is based on custom GLSL vertex and fragment shaders, manual buffer creation, texture management, lighting uniforms and transformation matrices.

Some basic support files and utility functions were taken from the exercise folder and from code examples shown during the course. These files provide the basic infrastructure for creating the WebGL context, compiling shaders, handling matrices and managing common rendering operations.

The rest of the project was extended with custom scene logic, custom shaders, animations, physics interactions, UI controls and game-like behaviors.

Cannon.js is used for the physics simulation of the ball and for the cloth-like curtain behavior.

External 3D assets are used for some models, while several objects are procedurally created or manually assembled inside the project.

Compatibility and performance

The project is designed to run in modern browsers with **WebGL 1.0** support. For the best experience, I recommend using Google Chrome with GPU acceleration enabled.

The target performance is **60 FPS**, optimized for displays with a maximum refresh rate of **60 Hz**. Since the project includes real-time shadows, physics, skinned animation and imported 3D assets, performance may vary depending on the device and browser.

A Performance Saver mode is available from the start settings panel to make the scene lighter on slower devices.

Project Structure

I organized the project into several files and each of them groups a specific part of the application logic. In the repository you can find:

- **index.html**: contains the HTML structure, UI elements and shader code.
- **style.css**: defines the layout, start screen, control panels and overlay panels.
- **main.js**: manages the main initialization, rendering loop, UI events, scene switching and global logic
- **config.js**: contains global variables, constants, asset paths and general project settings.
- **home_scene.js**: contains the rendering and interactions of the home environment.
- **park_scene.js**: contains the rendering and interactions of the park environment.
- **dog.js**: manages the skinned dog model, dog animation and dog behavior.
- **physics.js**: contains the Cannon.js physics setup for the ball and curtain simulation.
- **frisbee.js**: manages the frisbee interaction in the park scene.
- **shadow_maps.js**: contains the shadow mapping setup and shadow rendering passes.
- **utils.js**: contains general helper functions used for loading assets, textures and geometry.
- **utils_park.js**: contains helper functions used mainly for the park scene.
- **gltf_debug_loader.js** : was used to inspect and debug the imported GLB dog model and its skeleton.
- **sounds.js**: manages background music, sound effects and global audio controls.
- **README.md**: contains the full project documentation, credits, controls and references.
- **dog_skeleton_graph.png**: image file containing the generated dog bone hierarchy graph.
- **dog_skeleton_mermaid.txt** : text file with Mermaid code for the dog skeleton hierarchy diagram.

The repository also contains the following folders:

- **Textures**: contains the texture files used for objects, materials, skyboxes and effects.
- **Audio**: contains the music and sound effects.
- **Icons**: contains the icons used in the user interface.
- **Objects**: contains the imported 3D models used in the project.
- **Common**: contains the WebGL support files from the course material.
- **README_assets**: contains screenshots and images used here in the manual or in the README.

Scene Structure

There are two main scenes: the **Home scene** and the **Park scene**.

The **Home scene** contains the room environment, walls, floor, table, teapot, bowl, curtain, window, wall lamp and the main virtual pet interactions. It is the main scene for feeding, giving water, calling the dog, petting the dog and playing with the ball.

The **Park scene** provides an outdoor environment with grass, a bench, environmental background, sun visualization, falling leaves, night fireflies and frisbee gameplay, making the scene feel more dynamic and alive.

The user can switch between the two scenes through the interface, while some interactions are enabled only in the appropriate scene.



home scene



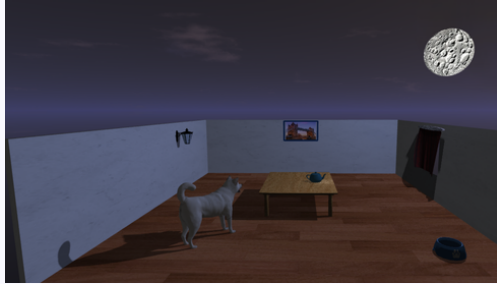
park scene

Rendering and Lighting

I developed custom **WebGL 1.0** shaders. In particular I used vertex and fragment shaders to manage object transformations, textures, normals and lighting. Most of the objects are rendered with a Blinn-Phong style lighting model, using ambient, diffuse and specular components.

The scene supports both day and night mode.

During the day, the main light represents the sun and can be moved with the sliders in the interface. I also added an automatic sun movement mode.



At night, the colors of the scene become colder and the moon is shown instead of the sun. In the home scene, the user can also turn on a wall lamp, which adds a warmer local light.

At night, the park also includes procedural fireflies to make the environment feel more alive.

Shadow Mapping

For the shadows, I wanted to challenge myself by implementing **shadow mapping**.

The shadow system works in two main steps. First, the scene is rendered from the point of view of the light, and the depth information is stored inside shadow textures. Then, during the normal rendering pass, the fragment shader compares the current fragment depth with the saved depth value. In this way, the shader can decide if that point is lit or in shadow.

There are two possible shadow mapping approaches. A directional shadow map is usually used for sun-like lights, where all light rays are treated as parallel and one shadow map is enough. A point-light shadow map is used for local lights that emit in many directions, so the scene must be rendered from multiple light directions.

The **main light** represents the sun/moon in the scene, but its shadow system is implemented with a point-light style approach. Instead of using a real cubemap, I used six 2D shadow maps, one for each direction around the light. This allows the main light to cast shadows in multiple directions in both the home and park scenes.

For **the wall lamp**, I added one separate 2D shadow map, used only when the lamp is active at night in the home scene.



Shadow map from main light



Shadow map from wall lamp

To make the result softer and reduce artifacts, I also used multiple shadow samples and small bias values.

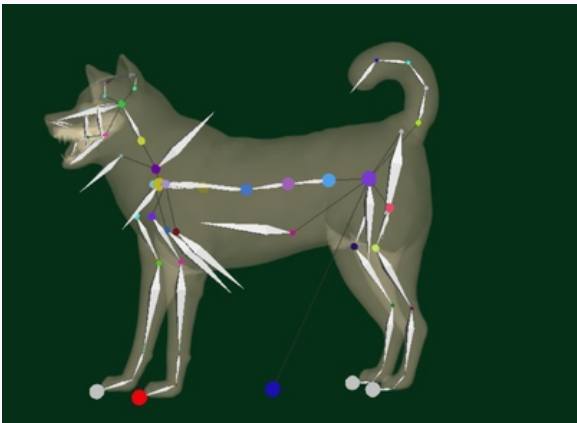
The dog model

The dog is the main animated character of the project. I used an imported rigged/skinned 3D model in GLB format, which is the binary version of the glTF format. The original source of the model is reported in the model references section.

The mesh is controlled by an internal skeleton, so the dog is animated through bones instead of being moved as separate rigid parts. The skeleton is organized as a hierarchy of bones. This means that some bones are children of other bones: for example, the spine controls the main body, while the neck, head, legs and tail depend on it. This structure makes the animation more natural, because moving a parent bone also affects the connected child bones.

The dog mesh uses skinning weights. Each vertex of the model can be influenced by one or more bones, and this allows the surface of the dog to deform smoothly during movement. In the visualization, the colored areas show how different parts of the mesh are connected to different bones.

In the project, I use the bone hierarchy to animate the dog in different situations: walking, moving toward a target, wagging the tail, eating, drinking, being called by the user and reacting during interactions.



Dog skeleton visualization: the rig controls the main body parts such as spine, head, legs and tail.



Skinning / bone influence visualization: different colors show how the mesh is affected by the skeleton.

The dog behavior is connected to the user interactions. When the user gives food or water, the dog moves toward the bowl and plays the correct animation. When the call mode is active, the user can click on the floor and the dog moves toward that position. In the park scene, the dog can also react to the frisbee interaction and run toward it.

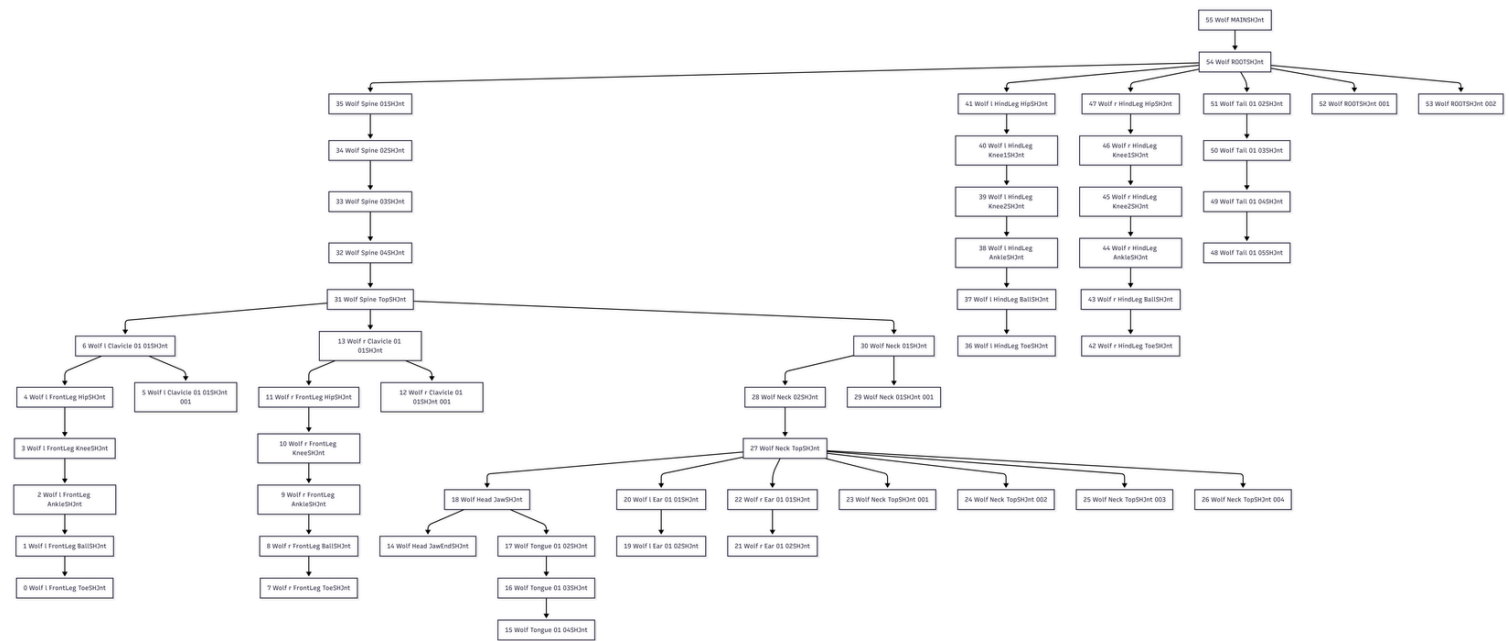


Dog in a pose specifically adapted to pick up the frisbee from the ground in park mode



The dog was placed on its stomach on the floor in home mode when it grabbed the ball

Dog Bone Hierarchy



Bone hierarchy graph generated from the dog rig structure using Mermaid.

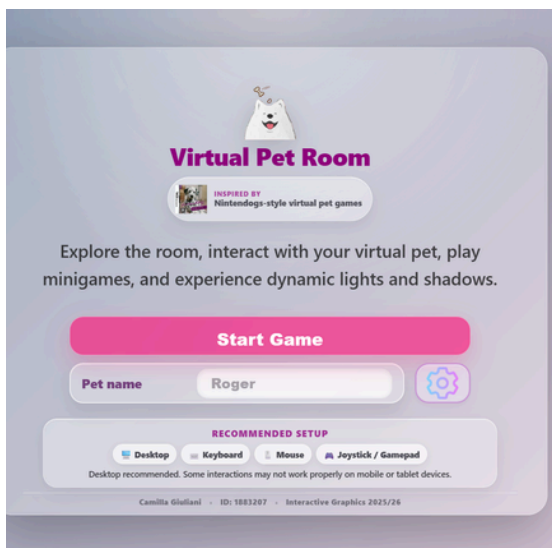
The dog skeleton is composed of **56 nodes**, corresponding to the bones and joints of the rig. To better understand the structure of the model, I analyzed the imported GLB/gITF file and reconstructed the parent-child relationships between the bones.

From this analysis, I generated Mermaid code and used a Mermaid diagram editor to visualize the hierarchy as a graph. In the graph, each node represents a bone or joint, while the arrows show how the bones are connected inside the skeleton.

This helped me select the bones needed for the main dog behaviors, such as walking, tail movement, eating/drinking poses and interaction reactions.

The gaming experience begins on this screen, where the user can access the game by clicking “Start Game” and can also name the little dog.

User Interface and Controls



The start screen introduces the user to the virtual pet experience before entering the 3D scene. From this screen, the user can start the game, choose a custom name for the dog and access the settings panel. It gives a quick overview of the expected interaction devices, such as keyboard, mouse and joystick/gamepad, so the user immediately understands that the project is designed as an interactive experience rather than a static 3D scene.

I added the pet name input to make the interaction more personal: the selected name is then used inside the interface in the Call Dog button.

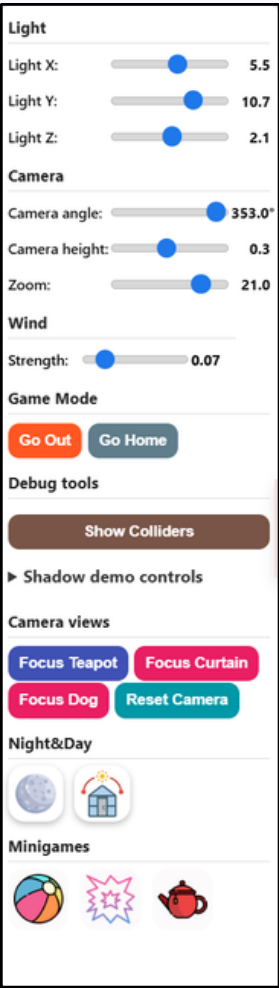
Once past the initial screen, the user is presented with a page where they can switch between “Home” and “Park” as mentioned above, but the layout remains the same with regard to the two elements at the top and on the left: a top control bar and a side control panel.



Top control bar screenshot

The top bar contains the most important interaction buttons, such as global audio, background music, water, food, call dog mode, pet dog mode and camera help. These controls are always easy to reach because they are the main actions that the user can perform during the game.

The side panel contains the more technical controls.

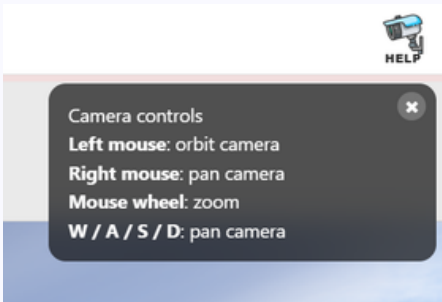


The user can change the light position with the Light X, Light Y and Light Z sliders, adjust the camera angle, height and zoom, control the wind strength and switch between the Home and Park scenes.

I also added buttons for different camera views, such as Focus Teapot, Focus Curtain, Focus Dog, Dog Camera and Reset Camera. This makes it easier to explore the scene and focus on specific objects or interactions.

The interface also includes day/night controls, automatic sun movement, the wall lamp button and the minigame buttons for the ball, teapot smash preset, teapot chase and frisbee.

Some interactions open small help panels. For example, the camera help panel explains how to orbit, pan and zoom, while the teapot and frisbee panels show the controls needed for those specific modes.



Side panel with technical controls (left) and camera help (above) overlay showing mouse and keyboard navigation.

Implemented interactions & animations

The project includes several real-time interactions between the user, the dog and the environment.

The user can **give water or food to the dog**. When one of these actions is activated, the bowl is filled and the dog moves toward it. Food and water are mutually exclusive, so if one is already active and the other one is selected, the previous one is removed.

The user can also **call the dog**. When Call Dog mode is active, the user can click on the floor and the dog moves toward the selected position.

Pet Dog mode allows the user to interact more directly with the dog and triggers a different reaction state.

The **ball minigame** can be started from the interface. When it is active, the ball is launched using physics parameters such as velocity, angular velocity, bounce, friction and damping. The dog can react to the ball and move toward it. We'll discuss this in the next section on physics part.

I also added a **teapot smash demo**. This button does not directly break the teapot: it starts the ball minigame with specific settings, so the teapot can be broken through the ball collision.

In the park scene, the user can play with the **frisbee**. The mouse is used to aim the hand, and a click throws the frisbee. The dog reacts by running toward it.

In night mode, the park scene also includes small procedural **fireflies**, which add a simple environmental animation and make the outdoor scene feel more alive. The dog stands up on its hind legs, trying to catch them

Another one is the **Teapot Chase mode**, where the user can control the teapot with keyboard or gamepad inputs while the dog follows it, making the interaction more similar to a small playable minigame.

To make the dog movement more believable, I did not simply teleport the dog to the target. I implemented a simple target-based movement and path system: when the user calls the dog, or when an object becomes interesting for the dog, the dog moves step by step toward that position.

I also added collider logic for some scene objects, such as the table (bowl and bench), so the movement and physics interactions can better respect the environment. A debug option can be enabled from the interface to show the colliders during development.

These elements are connected to the physics part of the project, which manages object collisions, ball behavior and some procedural movements in the scene.

Teapot Chase Controls

Keyboard:

I / J / K / L: move the teapot.
Arrow keys: rotate the teapot.
Q / E: lower or raise the teapot.
R: toggle teapot rotation.

Gamepad:

Left stick: rotate the teapot.
Right stick: move the teapot.
LT / RT: lower or raise the teapot.
A: toggle teapot rotation.



Animation of a dog bending down over its bowl to drink

Physics and minigame controls

I chose **Cannon.js** for the physics-based interactions because it fits well with a project developed directly in **WebGL**. Since I did not use a high-level rendering engine, **Cannon.js** allowed me to keep the rendering part fully custom while adding a separate physics simulation for collisions and dynamic objects.

The main physics elements are the ball simulation and the cloth-like curtain movement. The visual objects are still rendered by my **WebGL** code, while **Cannon.js** manages the physical behavior behind them.

For the **ball minigame**, I added a settings panel that allows the user to change the physical behavior of the ball before launching it. The user can control the initial velocity, angular velocity, bounce, friction, linear damping and angular damping.

This makes the ball interaction more flexible, because different values produce different trajectories and collision behaviors. The ball can also collide with the teapot and trigger the teapot break event.



The **curtain** near the window is also animated with a cloth-like behavior. Its movement is affected by the wind strength slider, so the user can increase or reduce the wind effect.

Ball Minigame Controls

Ball button: start or stop the ball minigame.

Vel X / Vel Y / Vel Z sliders: change the initial throw velocity of the ball.

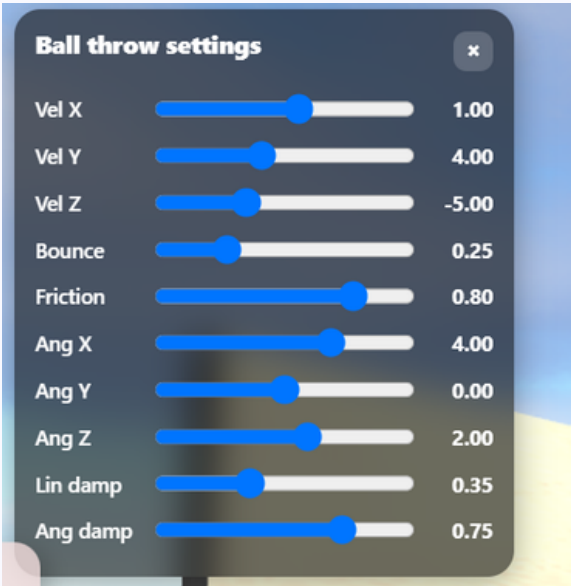
Bounce slider: changes how much the ball bounces.

Friction slider: changes how much the ball slows down on contact.

Ang X / Ang Y / Ang Z sliders: change the initial angular velocity of the ball.

Linear damping slider: controls how quickly the ball loses linear speed.

Angular damping slider: controls how quickly the ball stops rotating.



Ball Launch Settings Panel

Assets and references

3D Models Sources

- **Procedurally Generated and Code-Created Geometry**

The following scene elements are generated directly in code using custom buffers, primitive geometry, procedural meshes or runtime simulation.

Room structure

Description: Floor, back wall and left wall generated from box buffers and transformed with model matrices.

Right wall with window

Description: Custom wall mesh generated with a window opening.

Painting and frame

Description: Built from box geometry; the image and frame use textures.

Water surface

Description: Procedural disk mesh used to represent the water inside the bowl.

Kibble pieces

Description: Procedurally generated food particles.

Curtain cloth

Description: Runtime cloth mesh animated through Cannon.js physics.

Curtain rod / support

Description: Procedural support geometry generated in code.

Sun halo

Description: Billboard quad generated from manually defined vertices and texture coordinates.

Fireflies

Description: Procedural sphere meshes with generated color texture.

Debug colliders

Description: Simple generated helper geometry used to visualize physics colliders.

- **Third-Party and Imported 3D Models**

*All third-party models remain the property of their respective authors and marketplaces.
Some assets were purchased or downloaded under the license terms provided by the original platform.
They are included in this repository only for academic evaluation and university exam grading.*

Kishu Inu Dog , source CGTrader

Usage: Purchased rigged/skinned dog model used as the main pet character.

<https://www.cgtrader.com/3d-models/animal/mammal/kishu-inu-japan-dog-breed>

Grass Patch, source CGTrader

Usage: Purchased grass asset used in the park scene.

<https://www.cgtrader.com/3d-models/plant/grass/photorealistic-grass-with-patch-generator-and-animatio>

Moon, source CGTrader

Usage: Imported model used for night mode visualization.

<https://www.cgtrader.com/items/6369973/download-page>

Sun, source CGTrader

Usage: Imported model used for day mode visualization.

<https://www.cgtrader.com/items/5092264/download-page>

Frisbee, source CGTrader

Usage: Purchased model used in the frisbee minigame.

<https://www.cgtrader.com/3d-models/sports/toy/flat-disc-frisbee>

Leaf, source CGTrader

Usage: Imported model used for park/environment details.

<https://www.cgtrader.com/free-3d-models/plant/leaf/tree-leaf>

Bench, source CGTrader

Usage: Imported model used in the park scene.

<https://www.cgtrader.com/free-3d-models/exterior/street-exterior/park-bench-modern-outdoor-bench-long-seat>

Music Note, source CGTrader

Usage: Imported model used as dog feedback/interaction effect.

<https://www.cgtrader.com/free-3d-models/various/various-models/music-note-33d9edfd57a7bc1b72f2d9dedbea46e5>

Wall Lamp, source CGTrader

Usage: Imported model used for the night wall lamp and shadow demo.

<https://www.cgtrader.com/free-3d-models/exterior/street-exterior/wall-lamp-35a16b45-0427-4a69-907c-709c1568032d>

Dog Bowl, source CGTrader

Usage: Imported model used for food and water interactions.

<https://www.cgtrader.com/items/6235171/download-page>

Ball, source Free 3D

Usage: Imported model used in the ball minigame.

<https://free3d.com/it/3d-model/beach-ball-v2--259926.html>

Table, source Free 3D

Usage: Imported model used in the home scene.

https://free3d.com/3d-model/table-396579.html?dd_referrer=

Heart, source Free 3D

Usage: Imported model used as pet interaction feedback.

<https://free3d.com/3d-model/heart-v1--539992.html>

Teapot, source: Interactive Graphics Course material

Usage: Course-provided model used for rendering and minigame interaction.

Textures

I used a combination of imported texture maps, local texture files, external image textures and procedurally generated colors.

- **Imported Model Texture Folders**

Some texture maps are included together with the corresponding imported 3D models and are stored locally in the project folders.

Kishulnu_tex : texture maps for the rigged dog model.

Bench_tex : bench material maps used in the park scene.

Bowl_tex: bowl material maps, including normal and roughness details.

Table_tex: table material maps, including color, specular and ambient occlusion maps.

WallLamp_tex: wall lamp material maps, including base color, normal and roughness maps.

Grass patch: grass asset texture used for the park vegetation.

Sun: texture maps included with the imported sun model.

Leaf: texture maps included with the imported leaf model.

- **Procedurally Generated Colors and Effects**

Some visual elements do not rely on image files, but are generated directly in code using solid colors, alpha values or procedural geometry.

Teapot color: generated with a solid-color texture.

Teapot break fragments : procedural break animation triggered when the ball hits the teapot during the ball minigame.

Water surface: generated with a transparent blue solid-color texture and applied to a procedural disk mesh.

Kibble pieces: generated with a solid brown color and procedural geometry.

Fireflies: generated with a solid yellow color and sphere geometry.

Sun halo billboard: procedural billboard quad combined with a halo texture, alpha blending and a small pulse animation.

Debug colliders : rendered with simple generated helper geometry and transparent/debug colors.

• Skybox Textures

All skybox textures used in the project are from [OpenGameArt](#).

I used different skybox backgrounds for the indoor scene and the outdoor park scene. The home scene has separate day and night skybox textures, while the park scene reuses the day skybox and adapts it visually through the night lighting setup

Home / Day skybox

Usage: Background environment for the indoor scene during day mode.

Home / Night skybox

Usage: Background environment for the indoor scene during night mode.

Park skybox

Usage: Background environment for the outdoor park scene.

In night mode, this skybox is reused and modified through lighting.

• Additional Image Textures

Some image textures were manually collected during development and stored inside the project folders. They are used only to support scene materials, decorative elements and visual effects for this academic project.

The original source links were not tracked, therefore these images should be considered third-party reference assets and not original project assets.

Room materials

Used for: Walls, floor and curtain materials in the home scene.

Texture files: [wall_tex.jpg](#), [parquet_tex.jpg](#), [curtain_tex.png](#)

Decorative images

Used for: Back-wall picture and start screen inspiration badge.

Texture files: [london.jpg](#), [nintendogs_logo.jpg](#)

Lighting and sky effects

Used for: Moon marker and glow/halo visual effects.

Texture files: [moon_2.png](#), [halo.png](#)

Park vegetation

Used for: Vegetation details in the park scene.

Texture files: [grass_3.jpg](#)

Object textures

Used for: Extra textures used for interactive or scene objects.

Texture files: [frisbee_2.png](#), [ball_diff.jpg](#), [table_tex_512.jpg](#), [teapot_tex_1.png](#)

Utility textures

Used for: Simple color/material textures used for visual support or debugging.

Texture files: [black.jpg](#), [blue_navy.jpg](#), [red.jpg](#), [hot_pink.jpg](#)

Sounds

All sound effects and background audio tracks used in this project are from **Pixabay**.

Water: used for the water interaction and bowl filling sound.

<https://pixabay.com/sound-effects/film-special-effects-bubble-sound-effect-1-527144/>

Ball throwing: used as the ball throw sound effect.

<https://pixabay.com/sound-effects/film-special-effects-movement-swipe-whoosh-3-186577/>

Background Home: used as background music for the home scene.

<https://pixabay.com/music/beats-relax-summer-397430/>

Background Park Night: used as night ambience for the park scene.

<https://pixabay.com/sound-effects/nature-deep-south-night-sounds-115466/>

Background Park Day: used as day ambience for the park scene.

<https://pixabay.com/sound-effects/nature-020188-bird-park-ambience-74610/>

Pouring Kibbles: used for the food pouring interaction.

<https://pixabay.com/sound-effects/household-pouring-cornflakes-in-bowl-138367/>

Frisbee Whoosh: used as the frisbee throw sound effect.

<https://pixabay.com/sound-effects/film-special-effects-whoosh-1-522923/>

Dog Happy Jingle: used as positive dog feedback sound.

<https://pixabay.com/it/sound-effects/film-ed-effetti-speciali-notification-bell-361840/>

Dog Breath: used as the dog breathing sound effect.

<https://pixabay.com/sound-effects/nature-dog-breathing-5-fx-308616/>

Dog Bark: used as the dog bark sound effect.

<https://pixabay.com/it/sound-effects/film-ed-effetti-speciali-friendly-big-dog-bark-1-535471/>

Dog Drinking: used for the dog drinking interaction.

<https://pixabay.com/it/sound-effects/film-ed-effetti-speciali-dog-drinking-water-5-309519/>

Dog Eating: used for the dog eating interaction.

<https://pixabay.com/es/sound-effects/naturaleza-chewing-dog-eats-crunchy-crackers-29627/>

Wind: used with the curtain and park atmosphere.

<https://pixabay.com/sound-effects/nature-gust-of-wind-511325/>

Dog Kibble Interaction: used as additional feedback sound for the dog/food interaction.

<https://pixabay.com/sound-effects/film-special-effects-jump-sound-531048/>

Glass Breaking: used as the teapot breaking sound effect when the ball hits the teapot.

<https://pixabay.com/sound-effects/search/break%20glass/>

Mouse Click: used as UI/start button click sound.

<https://pixabay.com/sound-effects/film-special-effects-mouse-click-290204/>

Icons

Most UI icons are from **Flaticon** except for the Kishu Inu reference icon which is from **Pinterest**.

Kishu Inu Icon: Dog-themed reference icon.

<https://it.pinterest.com/pin/56295064084237515/>

Settings: for Settings button.

https://www.flaticon.com/free-icon/setting_2040510?term=settings&page=1&position=58&origin=search&related_id=2040510

Call Dog Hand: for Call dog interaction.

https://www.flaticon.com/free-icon/open-hand_889822

Wave Hand / Caress: Pet dog interaction.

https://www.flaticon.com/free-icon/wave_9606501?term=wave+hand&related_id=9606501

Open Hand Frisbee: Frisbee interaction state.

https://www.flaticon.com/free-icon/five-fingers_9971672?term=open+hand&related_id=9971672

Hand Holding Frisbee: Frisbee holding / throwing state.

https://www.flaticon.com/free-icon/grab_1196462

Mute / Audio Off: Audio disabled icon.

https://www.flaticon.com/free-icon/mute_561228

Auto Moving Sun: Automatic sun movement button.

https://www.flaticon.com/free-icon/greenhouse_4772346

Frisbee: Frisbee button.

https://www.flaticon.com/free-icon/frisbee_7601483?term=frisbee&page=1&position=23&origin=search&related_id=7601483

Music On: Music enabled icon.

https://www.flaticon.com/free-icon/musical-note_2995101?term=music+notes&page=1&position=9&origin=search&related_id=2995101

Music Off: Music disabled icon.

https://www.flaticon.com/free-icon/music-off_13407074?term=music+off&page=1&position=29&origin=search&related_id=13407074

Ball: Ball minigame button.

https://www.flaticon.com/free-icon/beach-ball_3012458?term=ball&page=1&position=8&origin=search&related_id=3012458

Dog Water Bowl: Water bowl interaction.

https://www.flaticon.com/free-icon/dog-bowl_6004496?term=dog+bowl+water&page=1&position=7&origin=search&related_id=6004496

Dog Food Bowl: Food bowl interaction.

https://www.flaticon.com/free-icon/dog-food_8876508?term=bowl+dog&page=1&position=8&origin=search&related_id=8876508

Sun: Day mode icon.

https://www.flaticon.com/free-icon/sun_10484062?term=sun&page=1&position=4&origin=search&related_id=10484062

Moon: Night mode icon.

https://www.flaticon.com/free-icon/full-moon_9689786?term=moon&page=1&position=4&origin=tag&related_id=9689786

Audio On: Audio enabled icon.

https://www.flaticon.com/free-icon/volume_10628912?term=audio&related_id=10628912

Teapot Icon: Teapot chase minigame.

https://www.flaticon.com/free-icon/teapot_491609?term=teapot&page=1&position=4&origin=search&related_id=491609

Camera Icon: Camera / focus controls.

https://www.flaticon.com/free-icon/cctv-camera_2642651?term=camera&page=1&position=23&origin=search&related_id=2642651

Paw Icon: Pet / dog UI decoration.

https://www.flaticon.com/free-icon/paw_18548576?term=paw&page=4&position=90&origin=search&related_id=18548576

Light Bulb: Wall lamp button.

https://www.flaticon.com/free-icon/idea_9901812?term=lamp&related_id=9901812

Damage: Teapot smash preset / break interaction button.

https://www.flaticon.com/free-icon/damage_7037237?term=damage&related_id=7037237